MISB MOTION IMAGERY STANDARDS BOARD	MISB ST 1902.3
STANDARD	
Motion Imagery Metadata (MIMD): Model-to-KLV Transmutation Instructions	**2025-06-11**

1 Scope

The Motion Imagery Metadata (MIMD) documents are composed of the MIMD Modeling Rules, MIMD Model, and Model-to-KLV Transmutation Instructions. MISB ST 1901 [1] defines the modeling rules which are a subset of standardized UML class diagrams specifically minimized to enable transforming, or transmuting, class instances to KLV. MISB ST 1903 [2] defines the MIMD Model, which is a UML model following the rules from MISB ST 1901 and organized as a hierarchy of information including metadata for temporal, platform, payload, sensor, command, automated processes, exploitation, security, and more. This standard defines the Model-to-KLV Transmutation Instructions for constructing bandwidth-efficient KLV structures from the model. Other formats such as XML or JSON may have transmutation instructions developed for them in future revisions. This document, in concert with MISB ST 1901, MISB ST 1903 and other MIMD model detail documents, provides a suite of standards embodying the MIMD architecture.

2 References

[1] MISB, *ST 1901.3 - Motion Imagery Metadata (MIMD): Modeling Rules,* 2025-06-11.

[2] MISB, *ST 1903.3 - Motion Imagery Metadata (MIMD): Model,* 2025-06-11.

[3] MISB, *ST 0107.5 - KLV Metadata in Motion Imagery,* 2021-10-21.

[4] MISB, *MISP-2025.1: Motion Imagery Handbook,* 2025-06-11.

[5] SMPTE, *ST 336:2017 - Data Encoding Protocol Using Key-Length-Value.*

[6] MISB, *ST 1904.3 - Motion Imagery Metadata (MIMD): Base Attributes,* 2025-06-11.

[7] ITU-T, *X.690 - Information Technology - ASN.1 Encoding Rules - Specification of Basic Encoding Rules (BER), Canonical Encoding Rules (CER) and Distinguished Encoding Rules (DER),* 2021-02.

[8] ISO/IEC, *10646:2020 - Information Technology - Universal Coded Character Set (UCS).*

[9] IEEE, *754-2019 - Standard for Floating-Point Arithmetic.*

[10] MISB, *ST 1303.2 - Multi-Dimensional Array Pack,* 2020-06-25.

[11] MISB, *ST 1201.5 - Floating Point to Integer Mapping,* 2021-06-24.

[12] MISB, *ST 0807.27 - MISB KLV Metadata Registry,* 2025-06-11.

3 Revision History

Version	Approved Date	Change Description
ST 1902.3	2025-06-11	• Changed the method of transmuting lists to use an Administration Pack which removes the list type vagueness (this is a breaking change, so there is a new KLV MIMD root key for this version) ○ The Administration Pack includes the list type and other information ○ Included transmutation instructions for both types of lists ○ Added requirements -11 & -12 • Invokes ST0107 for: ○ Report-on-Change, via new requirement -10 ○ Transmutation of Strings ○ Transmutation of floats to IMAP • Added a Report-on-Change instruction for transmuting parent instances when transmuting an element in a child instance, i.e., transmute all child's parents whenever a child value changes • Added ZLI instructions for working with inheritance relationships • Added new KLV key for the MIMD root • Retired requirements -02, -03, -04, & 06 because they duplicate requirements in ST1901 • General editing ○ Editorial revisions to requirements -06, -07, -09 ○ Minor edits
	2025-11-30	• Editorial changes for the Admin Pack to use ALBA instead of BER-OID, as the admin pack value is not an object identifier (OID). There are no changes in data format or requirements.

4 Definitions of Terms, Acronyms, and Initialisms

4.1 Terms

The following lists the terms and their definitions this document uses.

Report-on-Change	A method to reduce bandwidth over time by only writing changed data in a data packet instead of writing pre-stated values in every packet; see Motion Imagery Handbook
ROC_Unknown	A Report-on-Change status indicating the state of an attribute is not known; see MISB ST 1901

4.2 Acronyms and Initialisms

The following lists the acronyms and initialisms this document uses.

APA	Array Processing Algorithm
ATS	Abstract Test Suite
DLP	Defined Length Pack
IEEE	Institute of Electrical and Electronics Engineers
ISO/IEC	International Organization for Standardization/ International Electrotechnical Commission
KLV	Key Length Value
MDAP	Multi-Dimensional Array Pack

MIMD	Motion Imagery Metadata
MISB	Motion Imagery Standards Board
MISP	Motion Imagery Standards Profile
ROC	Report-on-Change
UML	Unified Modeling Language
ZLE	Zero Length Element
ZLI	Zero Length Item

5 Introduction

The purpose of the MIMD suite of standards is to provide a framework that consolidates and organizes information into a singular, unified metadata model, thereby serving many use cases. This MIMD model then transmutes into different formats, such as KLV or XML.

This MIMD standard addresses the processes for generating efficient KLV (and other future formats) from the MIMD model. Figure 1 identifies the five components of the MIMD system: MIMD Modeling Rules, MIMD Model, MIMD Instance, MIMD Transmutation Instructions, and MIMD Formats. The blue items represent MISB standards, and the green items represent implementation data.

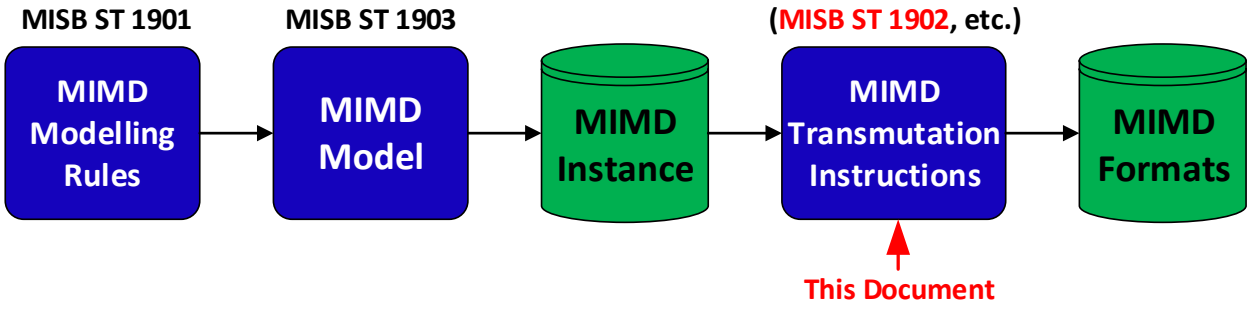


Figure 1: MIMD System Components

The MIMD Modeling Rules comprise a subset of the UML rules for building the MIMD Model. These rules enable transforming, or transmuting, MIMD Instances into MIMD Formats by the MIMD Transmutation Instruction documents.

The MIMD Model is a UML data model following the MIMD Modeling Rules. The Model contains classes, class attributes, and defines relationships between classes. MISB ST 1903 and its supporting documents define the MIMD Model.

A MIMD Instance is a data structure implementation of the MIMD Model. While a class defines the allowed contents and rules, an instance is a specific implementation of the class. In other words, a class is the blueprint; the instance is the actual data.

The MIMD KLV Transmutation Instructions provides the "algorithm" for converting a MIMD Model Instance into MIMD Formats. Currently, the only MIMD Format is KLV. MISB ST 1902 defines the MIMD Model-to-KLV Transmutation Instructions that map instances and their relationships to hierarchal KLV structures (e.g., Local Sets and Packs). Additionally, these instructions define methods for the "special" model functions for KLV-unique encodings (e.g.,

temporal compression). The MISB intends to develop other transmutation instructions for other formats such as XML or JSON as needed.

MIMD Formats are the final product of the MIMD system. The MIMD KLV Format is a bit-efficient implementation of the MIMD Instance data. Systems generate custom subsets of the MIMD KLV by selecting appropriate classes and class attributes for their system and following the MIMD Model-to-KLV Transmutation Instructions and requirements. Systems that create metadata and transmute it to MIMD Formats are producers. Systems that decode MIMD Formats and process the metadata are receivers.

6 MIMD Model-To-KLV Transmutation Instructions

The MIMD Model-to-KLV Transmutation Instructions are composed of two types: Conventional and MIMD Packet. Conventional instructions (see Section 6.1) provide details on the direct mapping of class instances and their relationships to KLV; most of the MIMD Model uses these instructions to convert to KLV. MIMD Packet instructions (see Section 6.3) provide details on how to create the KLV MIMD Packet and embed the MIMD or Root class instance, which contains all its "children" instances.

The KLV transmutation processes utilize the instructions and invoke the requirements in MISB ST 0107 [3]. Producers performing MIMD KLV transmutation may use Report-on-Change (ROC) from MIMD packet to MIMD packet, thus eliminating redundant information over time. The Motion Imagery Handbook [4] fully describes the concepts of the Report-on-Change method and ST 0107 provides Report-on-Change requirements. Receivers always assume producers are using Report-on-Change MIMD packets. Even though producers may choose not to use Report-on-Change, producers need to understand the Report-on-Change rules to know the impact of the receiver's data interpretation when a producer stops reporting data.

Requirement	
ST 1902.3-10	Producers shall adhere to the requirements for usage stated in MISB ST 0107.

6.1 Conventional Instructions

The Conventional instructions describe how to transmute the MIMD Model class instances to KLV. Each class instance independently transmutes to a KLV Local Set. A Composition relationship defines child classes, which transmute to child KLV Local Sets that embed into their parents Local Set. The child instances may have one or more children, which also transmute to embedded KLV Local sets, etc. This concept defines a recursive transmutation capability for Composition relationships.

Inheritance relationships merge their parent's attributes into the child class and become a single transmuted instance. Figure 2 illustrates a simple MIMD Model and its transmutation to KLV; details of the transmutation methods follow in subsequent sections. In this illustration, the MIMD class, or Root class defines an overall MIMD KLV Packet which is a Defined Length Pack (in blue). The defined length pack contains a Key/Length Check Value, the contents of the MIMD class, and a MIMD Local Set Check Value (see Section 6.3 for MIMD packet details). The contents of the MIMD class include: a version attribute, Platform class relationship, and a Timers class relationship. In a similar fashion, the Platform class is composed of more attributes and class relationships, re-creating the hierarchy of the class model through recursive transmutation.

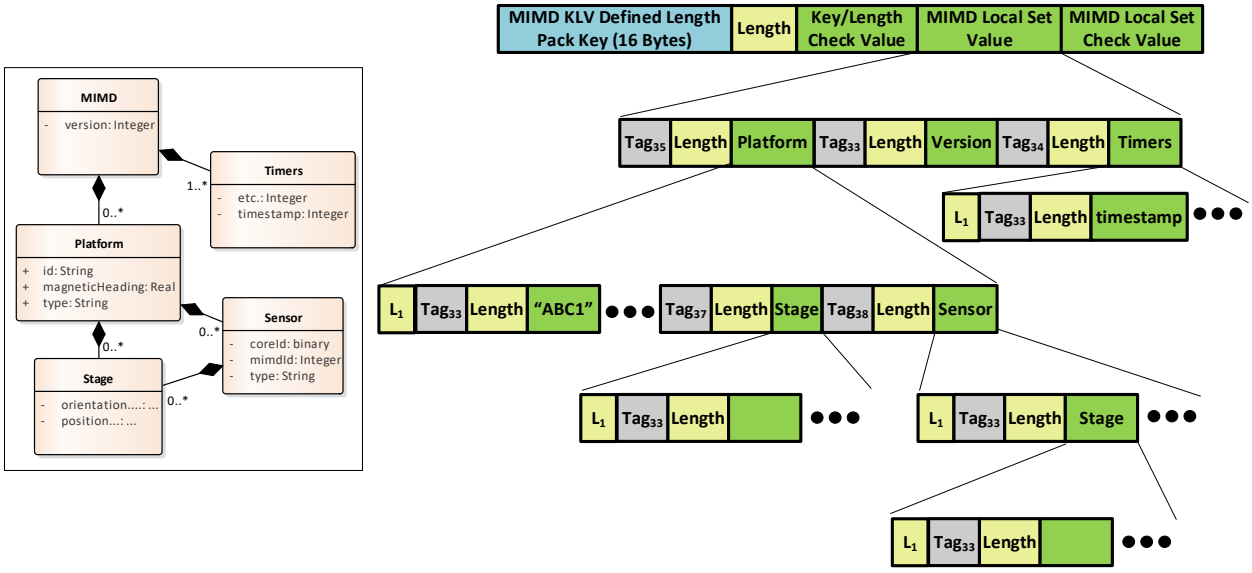


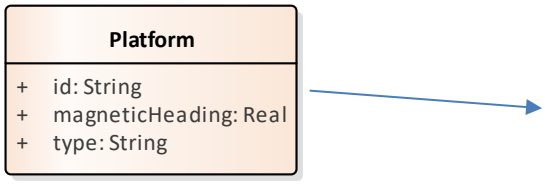
Figure 2:Illustration of Class to KLV Transmutation

The following Conventional instructions include sections on Single Class Instance Transmutation (i.e., with no relationships), Composition Transmutation, Inheritance Transmutation and Directed Association Transmutation.

6.1.1 Single Class Instance Transmutation

The MIMD modeling documents (e.g., MISB ST 1903) define classes, attributes within classes, and attribute traits (Name, Type, Identifier, etc.). When mapping class instances to KLV each class becomes a KLV Local Set. All MIMD Local Sets are consistent with a SMPTE 336 [5] Local Set defining byte 6 of its key equal to 0x0B, which indicates the Length field is ASN.1 BER short or long form and the Local Tag field is encoded using ASN.1 OID BER. Every attribute of a class becomes a Local Set item (see MISB ST 0107 [3]) with tag, length, and value. Section 6.1.1.1 provides details on transmuting attributes to KLV values.

Figure 3 illustrates an example model class and an instance of the class in tabular form with corresponding Local Set item's tags and values. The figure shows a UML Platform class on the left with three attributes (id, magneticHeading, and type) as it would appear in the MIMD Model. The right side shows a class instance in tabular form including associated tags and example values. Local Set Items 1 through 32 are "Reserved for Base Attributes", which are attributes every class includes MISB ST 1904 [6] provides details for the Base Attributes. In this example, the MISB pre-assigns Local Set Items 33, 34, and 35 to the class attributes within the defining document for the Platform class.



Local Set Items for Platform

Tag	Name	Value
1..32	Reserved for Base Attributes	Not used
33	Id	ABC1
34	magneticHeading	75.3
35	Type	PredatorB

Figure 3: Example of class instantiation

Figure 4 illustrates the result of transmuting the class instances of Figure 3 into KLV. The Platform Local Set begins with a tag (*Tag$_{xx}$*) from its parent Local Set, a total Length (*Length (Total)*), and the Local Set items prefixed with Tag 33 through Tag 35. Each Local Set item consists of a Tag (grey), Length (yellow), and Value (green). Each Length shows the value-length in parenthesis.

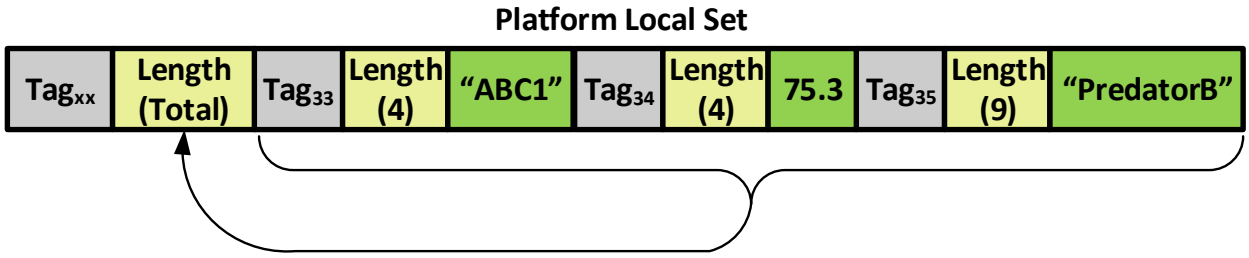


Figure 4: Class Instance to KLV

In cases where a system cannot supply the value for an attribute within its instance, the producer does not include it in the instance, and therefore it is not part of the KLV. For example, if the instance of the Platform class in Figure 3 did not have a value for magneticHeading, the KLV does not include a tag, length, and value for the magneticHeading attribute, as shown in Figure 5.

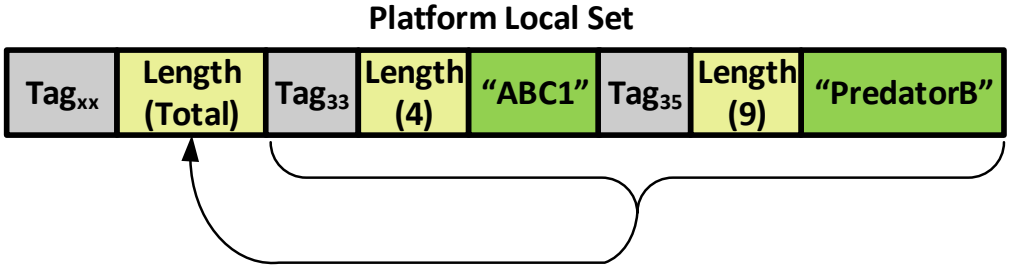


Figure 5: Class Instance to KLV w/o magneticHeading

When producers use Report-on-Change, if an instance's attribute does not change since the last MIMD Packet, the producer may choose not to include that attribute in the transmutation. If all attributes in a class instance do not need updating, a producer may choose not to transmute the whole class instance.

6.1.1.1 Transmuting Attributes to KLV Local Set Items

An attribute's traits determine its Local Set item's tag and data format. Figure 6 illustrates an attribute's traits (i.e., Name, Type, Identifier, Minimum, Maximum, DefaultResolution, and MaximumLength) and how they transmute to a single KLV Local Set item.

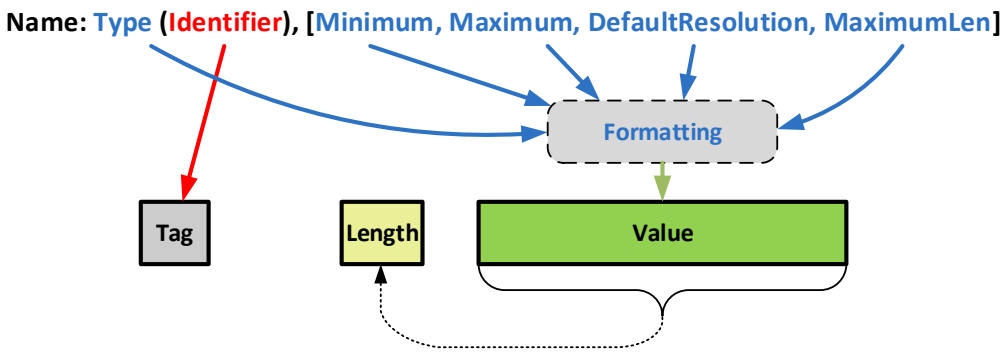


Figure 6: Attribute Traits to KLV Transmutation

A Local Set item's tag is the value of the Identifier trait, encoded as a BER-OID [7] value. The length is the size of the value (in bytes) encoded as a BER short or long form unsigned integer. The item's value is a binary representation of the class attribute's value with the format based on the attribute traits. KLV transmutation does not use the Name, Units, or Deprecate traits.

If an attribute does include a Deprecate trait the producer may not transmute the attribute into KLV.

Requirement	
ST 1902-01	Attributes including the Deprecate trait shall not be transmuted into KLV.

The attribute's traits plus the instance value itself provide the information for transmuting model values to a KLV Local Set value. The process of transmuting values may utilize trait information based on the Type trait. Table 1 lists the traits each data Type may use during transmutation. The Integer and UInt types are qualified by the Minimum and Maximum traits for validating the value is within the given range if provided; these traits do not provide information for Integer or UInt transmutation.

Table 1: Traits Each Type's Transmutation Uses

Type	Minimum	Maximum	DefaultResolution	MaximumLength
String	No	No	No	Yes
Real	Yes	Yes	Yes	No
Integer	Validation Only		No	No
UInt	Validation Only		No	No
Boolean	No	No	No	No

Local Set item values may be in the form of Singular Type or Array Type (see MISB ST 1901). The following sections provide transmutation instructions for a Singular Type. Section 6.1.1.2 provides the transmutation instructions for an Array Type.

6.1.1.1.1 Transmuting String

The MIMD Model values with the attribute type "String" transmutes to a Local Set value by converting the String value into a sequence of UTF-8 characters (see ISO/IEC 10646 [8]). The transmutation of the "String" type uses the instructions and requirements in MISB ST 0107 for "utf8" type. The size, in bytes, of the UTF-8 string becomes the KLV item length. If the attribute includes the MaximumLength trait, the number of characters in the string may not exceed the maximum length defined by the trait. The KLV item length has units of bytes, while the MaximumLength trait has units of characters; developers need to evaluate the correct length. A UTF-8 character has varying length with a minimum of one byte up to a maximum of four bytes; see the Motion Imagery Handbook for further information on "UTF-8" data types.

6.1.1.1.2 Transmuting Real

The MIMD Model attribute type "Real" transmutes using the instructions and requirements in MISB ST 0107 for "float" types. Based on the producer's accuracy and bandwidth requirements, the producer determines whether to use a single (4 bytes) or double precision (8 bytes) IEEE 754 [9] floating point value. The size, in bytes, of the floating-point value becomes the KLV item length. Receivers use the Local Set item's length to determine if the item's bit pattern is a single or double precision value.

Future versions of MISB ST 1902 may include binary16 (half precision) or binary128 (quad precision) from IEEE-754.

6.1.1.1.3 Transmuting Integer

The MIMD Model attribute type "Integer" transmutes to a Local Set item's value using the instructions and requirements in MISB ST 0107 for "int" values. The transmutation does not require Minimum or Maximum traits; however, the KLV values must not exceed these limits, if the MIMD Model provides them.

The length of the resulting value (i.e., with leading sign extension bytes trimmed) becomes the item length of the Local Set item.

6.1.1.1.4 Transmuting UInt

The MIMD Model attribute type "UInt" transmutes to a Local Set item's value using the instructions and requirements in MISB ST 0107 for "uint" values.

The transmutation does not require Minimum or Maximum traits; however, the KLV values may not exceed these limits, if the MIMD Model provides them.

The length of the resulting value (i.e., with leading zero bytes trimmed) becomes the item length of the Local Set item.

6.1.1.1.5 Transmuting Boolean

The MIMD Model attribute type "Boolean" transmutes to a single byte with either the value of zero (0x00) for false or one (0x01) for true; all other values are not valid. The Local Set's item length of the resulting value is always one (1).

6.1.1.1.6 Transmuting an Enumeration

An enumeration item transmutes using the value of the enumeration's numeric identifier. An enumeration numeric identifier (from the enumeration's table definition) within the MIMD

Model transmutes to its Local Set item's value using the instructions and requirements in MISB ST 0107 for "uint" values (ST 0107 uses a different capitalization for "UInt"). Although the transmutation does not utilize attribute Minimum or Maximum traits, the item's value needs to match one of the non-deprecated values in the enumeration's definition table.

Requirement	
ST 1902.1-08	The value of an encoded enumeration numeric identifier shall be a non-deprecated item value within the range of the enumeration's table definition.

The length of the resulting value (i.e., with leading zero bytes trimmed) becomes the item length of the Local Set item.

6.1.1.1.7 Transmuting a Tuple Type

A Tuple Type is a series of UInt values. The Tuple Type transmutes by serially encoding each element of the series as a BER-OID value. The combination of BER-OID values becomes the KLV item's value. This is an extremely efficient method of encoding a series of UInt values.

An example transmutation of a four element UInt series containing the values (100, 200, 400, 160000) is the combination of the four BER-OID values (0x64, 0x8148, 0x8310, 0x89E200), resulting in a single KLV item's value: 0x6481 4883 1089 E200. The KLV item's length for this example is eight bytes.

The decoding process repeatedly decodes individual BER-OID values from the KLV item's value without exceeding the KLV item's length. The result is a series of individual UInt values which is the Tuple's value.

6.1.1.2 Transmuting an Array Type

As defined in MISB ST 1901, the Array Type is a list of Singular Types arranged as a multi-dimensional array. The MIMD Model may define an Array Type with a specific number of elements (Defined Length Array), maximum number of elements (Maximum Length Array), or a flexible number of elements determined at run-time (Open Length Array). In all cases, systems transmute the Array Type instance into a series of individual Singular Type transmutations that share common encoding parameters, such as length. The notation of […] indicates unknown number of dimensions and unknown number of elements in each dimension.

MISB ST 1303 [10] defines the method of transmuting most Array Types to KLV using the Multi-Dimensional Array Pack (MDAP). Data producers determine which Array Processing Algorithm (APA) method, if any, at run-time and every instance may be different for each MIMD Packet and within a Packet.

6.1.1.2.1 Transmuting Real[…]

The MIMD Model attribute type "Real[…]" transmutes into an MDAP (as described in Section 6.1.1.2). There are two methods for applying MDAP for real values: IEEE and IMAP [11]. With the IEEE method each element transmutes into an IEEE 754 element (see Section 6.1.1.1.2). With the IMAP method, the producer computes the minimum and maximum of the array, along with a common length to use based on desired accuracy. The producer transmutes the values using the instructions and requirements in MISB ST 0107 for "IMAP" values. The producer sets the MDAP APA indicator for IMAP along with the APA Minimum and Maximum values. The producer applies the IMAP algorithm to each value in the array using the computed minimum, maximum, and chosen common length.

6.1.1.2.2 Transmuting Integer[…]

The MIMD Model attribute type "Integer[...]" transmutes into an MDAP (as described in Section 6.1.1.2). The producer pads each element to match the size of the largest element (by byte size) and the appropriate MDAP parameters are set.

6.1.1.2.3 Transmuting UInt[…]

The MIMD Model attribute type "UInt[...]" transmutes into an MDAP (as described in Section 6.1.1.2). Pad each element to match the size of the largest, by byte size, element and the appropriate MDAP parameters are set.

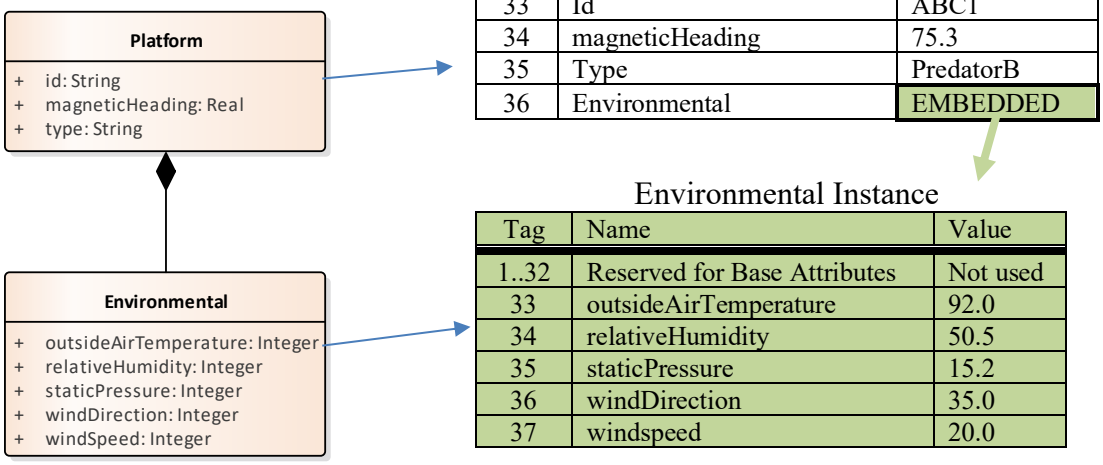
6.1.1.2.4 Transmuting Boolean[…]

The MIMD Model attribute type "Boolean[...]" transmutes into an MDAP (as described in Section 6.1.1.2).

6.1.2 Composition Transmutation

A Composition relationship in a class model becomes a single item in its parent class Local Set; the single item's value is the "value" portion of a Local Set with its constituent items.

Figure 7 illustrates the transformation of a Composition relationship of two MIMD classes to KLV. The illustration shows two model classes on the left, Platform and Environmental, and corresponding class instances on the right in tabular format. The Platform class consists of Base Attributes, Platform attributes, and relationship attributes. As in Figure 3, the Base Attributes and Platform attributes map to Local Set items 1 through 35; however, Item 36 maps to the Composition relationship of the Platform class to the Environmental class. The Platform class is the parent class, so it includes the item for the relationship to the Environmental child class.



Platform Instance

Tag	Name	Value
1..32	Reserved for Base Attributes	Not used
33	Id	ABC1
34	magneticHeading	75.3
35	Type	PredatorB
36	Environmental	EMBEDDED

Environmental Instance

Tag	Name	Value
1..32	Reserved for Base Attributes	Not used
33	outsideAirTemperature	92.0
34	relativeHumidity	50.5
35	staticPressure	15.2
36	windDirection	35.0
37	windspeed	20.0

Figure 7: Illustration of Composition relationship's class instantiation

When generating KLV from the instances, the Environmental Local Set embeds into the Platform's Local Set as Item 36. Figure 8 illustrates the resulting KLV generated using the

MIMD Model-To-KLV instructions. The Platform Local Set starts with a tag (Tag_{xx}) from its parent Local Set, a Length (Total), and the Local Set items identified by Tag_{33} through Tag_{36}. The Length (Total) is the combined length of all tags, lengths and values for the Platform and its embedded Environmental Local Set, as shown with the dotted line and arrow. The value for Item 36 is the Environmental Local Set mapping composed of Environmental Local Set Items 33 through 37.

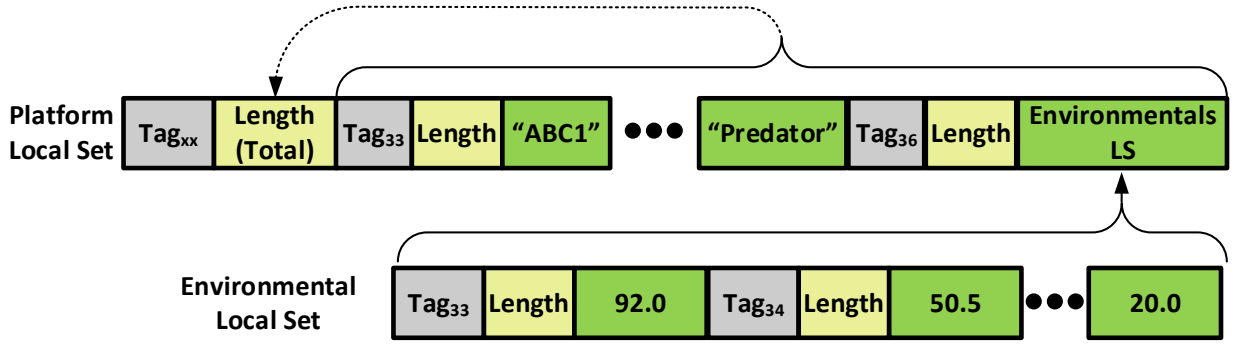


Figure 8:Example of a MIMD Class to KLV Local Set

Per ST1901, with Report-on-Change, transmuting an attribute in the child class forces the producer to also transmute the parent instance. The child's transmutation may only contain the one attribute and the parent transmutation may only contain the one child class. Likewise, the parent's transmutation forces its parent's (i.e., the grandparent) transmutation, all the way up the hierarchy to the Root class.

6.1.2.1 Class Multiplicity

With class multiplicity, a parent class relates to zero or more of the same child class. With multiplicity, the parent uses a list of elements where each element in the list is a single instance of the child class.

Figure 9 illustrates the multiplicity relationship of a Platform class with the Sensor class and how it translates to class instances.

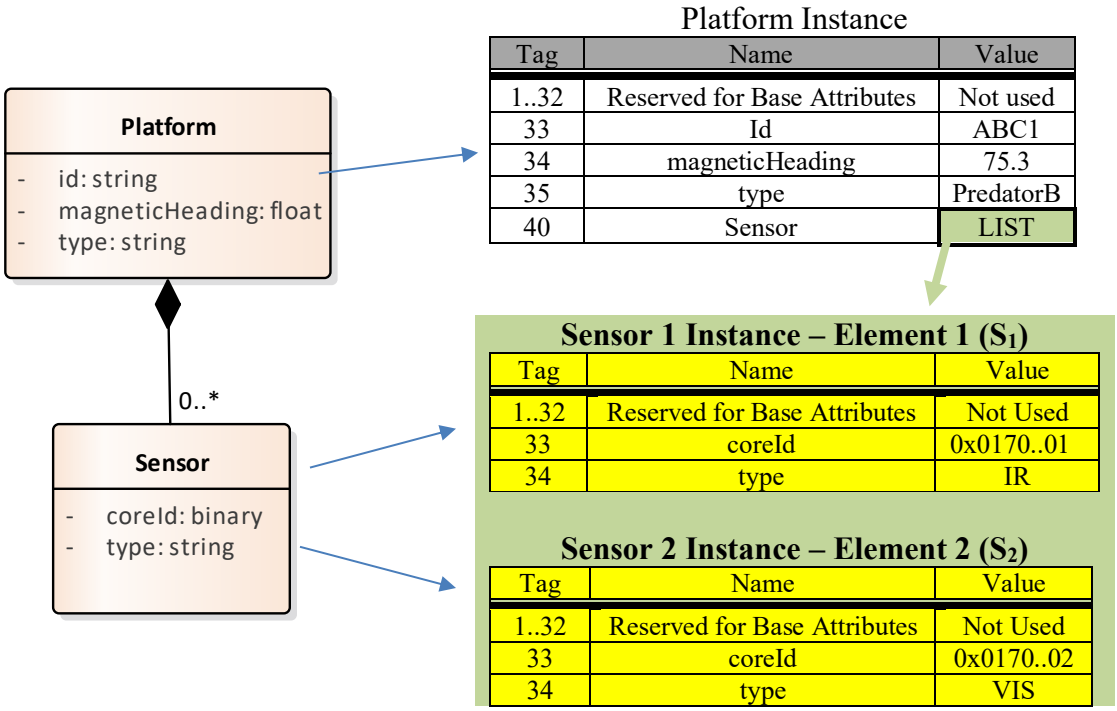


Figure 9: Illustration of Multiplicity

In this example, the Platform class maps to a Platform instance in the same fashion as in prior examples; however, the Sensor instantiations are elements of a list within the Platform instantiation, as shown in green. Nominally, the list's elements, S_1 and S_2, are not order dependent and they may each have different defined attributes (i.e., their sizes may be different).

MISB ST 1901 defines two methods for organizing Composition relationship lists, Fixed Order and Marked Element. Furthermore, with Report-on-Change, producers may need to mark list elements as ROC_Unknown. To indicate which organization method the list uses and to provide Report-on-Change information, each list includes an upfront List Administration Pack (Section 6.1.2.1.1 provides details).

The MIMD Model-To-KLV instructions define the use of a Defined Length Pack (DLP) for transmuting the List Administration Pack and the list elements. The DLP contains two elements, the List Administration Pack and a List Series. The List Series contains length-element pairs denoted by (L_i, E_i), where E_i is the ith list element transmuted into a KLV Local Set, and L_i is that Local Set's length (L_i) in bytes. With N list elements, the DLP contains the List Administration Pack (AdminPack) and a single List Series of N items, i.e., AdminPack, List Series = [(L_1, E_1), …, (L_i, E_i), ..., (L_N, E_N)].

Figure 10 illustrates the resulting KLV for the Platform class instance consisting of a multiplicity of Sensors. The Platform Local Set begins with a tag (*Tag$_{xx}$*) from its parent Local Set, a total length (*Length (Total)*), and Local Set Items 33, 34, 35 and 40. Item 40 contains the AdminPack and the list of Sensor Local Sets in a DLP (red dashed box). The length following Tag_{40} (Item 40's tag), is the size of the DLP which includes the sizes of both the AdminPack and List Series. The first element of the DLP is the AdminPack (purple) and the second element is the List Series (blue). Within the List Series, each list element starts with the length of the list element (e.g., L_1)

followed by element's Local Set items. This illustration shows the *Sensor Local Set Element 1* starting with *Length L_1* then the Sensor Local Set (S_1) elements follow (i.e., Item 33 – 0x0170..01 and Item 34 – "IR"). The AdminPack has a computable length allowing decoders to know where the List Series starts within the DLP. Within the List Series a length (L_i) precedes each Sensor Local Set (S_i) element allowing decoders to separately extract each list element.

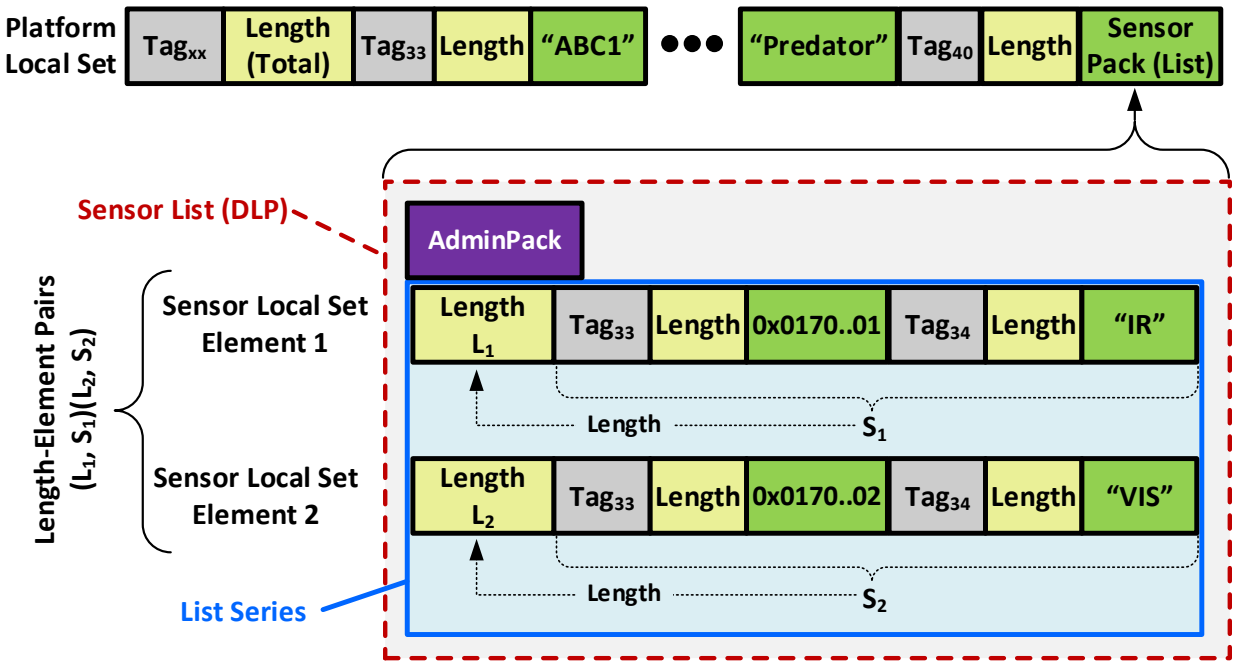


Figure 10: Example of MIMD KLV for a class with multiplicity

With Report-on-Change, when the producer chooses not to transmute all elements of a list, the producer may omit the list item in the parent Local Set. For example, if there is no change to the list of Platform classes (and the attributes of each Platform class), the producer may choose to omit the Platforms item in the MIMD class.

6.1.2.1.1 List Organization

Per ST 1901, there are two methods for organizing Composition relationship lists: Fixed Order and Marked Element. A producer declares the list organization method using the first element of the DLP, i.e., the List Administration Pack. The List Administration Pack contains a single Arbitrary Length Bit Array (ALBA) (see Appendix A) containing a bit-array which has the Admin Bit Flags and optionally a Report-on-Change bit mask (ROC Bit Mask). Figure 11 illustrates the Admin Bit Flags are the first seven bits (blue) of the bit-array and the remaining bits contain the optional ROC Bit Mask (pink).

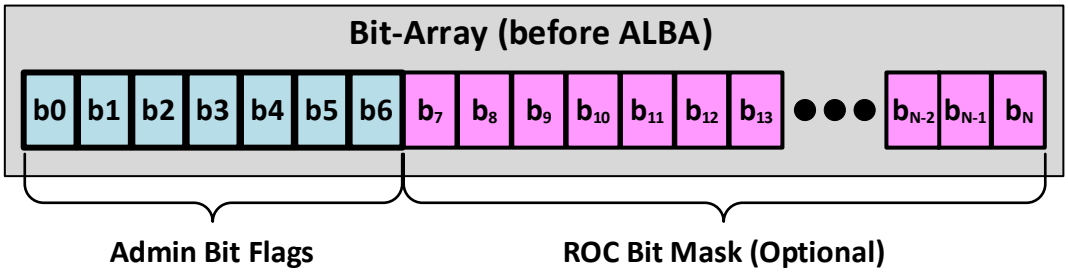


Figure 11: Admin bit-array before ALBA

Table 2 lists the Admin Bit Flags purpose. The first bit , b_0, indicates the list organization method, either Marked Element (zero) or Fixed Order (one); bit b_1 through b_6 are for future use and are set to zero.

Table 2: Admin Bit Flags

Bit Position	Meaning	Values
b_0	List Organization Method	Marked Element list = zero (0) Fixed Order list = one (1)
$b_1 – b_6$	Future	All set to zero (0)

The ROC Bit Mask declares which elements of the list are to become ROC_Unknown after a reader reads and processes the packet. If there are no elements to set to ROC_Unknown, the producer does not include the bit mask in the AdminPack. The ROC Bit Mask starts at b_7 where each bit is for a list series item in the order of the list series; b_7 is for list item 1, b_8 is for list item 2, etc., extending on until the last list item is set to one. Producers truncate the remaining bits representing the list items which would have a bit value of zero, i.e., the remaining bits are not included in the ROC Bit Mask. At most there can be up to one bit per list item.

When a producer has a list element which is ROC_Unknown, the producer sets the bit position for that element to one; a value of zero (default) means there is no change to the list element's ROC status. The ROC Bit Mask needs to be long enough to signal which elements are set to ROC_Unknown; for example, setting element three to ROC_Unknown in a packet-list with a 100 elements, only needs one byte for the ROC Bit Mask because all the other elements are not changing their ROC status.

Producers use ALBA (see Appendix A), to construct the List Administration Pack. Figure 12 illustrates an AdminPack (purple) example which equals a single ALBA bit-array. The first bit (red) is the ALBA continuation bit; in this example there are two bytes so the first ALBA continuation bit is set to one, and the second ALBA continuation bit is set to zero. ALBA bit-arrays can have many bytes to support long lists. The first byte of the ALBA bit-array contains the Admin Bit Flags (blue) per Table 2; the List Organization Method (orange) (b_0) indicates if the list is Marked Element or Fixed Order; the next six bits (grey) are for future use and set to zero. Starting with the second byte, if present, is the ROC Bit Mask (pink). This illustration

shows a single byte ROC Bit Mask value representing the ROC status for up to the first seven packet-list items.

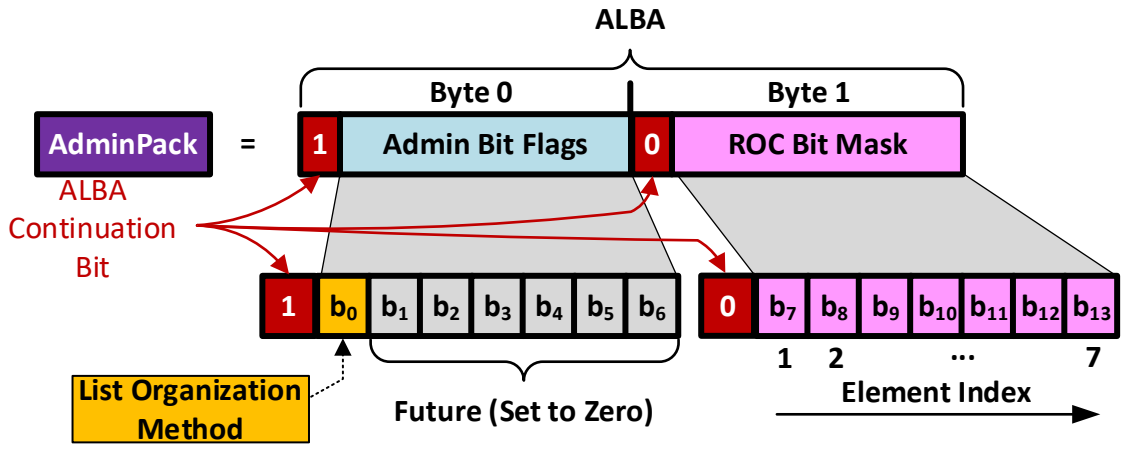


Figure 12: List Administration Pack

6.1.2.1.1.1 Marked Element Lists

A Composition relationship list where all list elements include a mimdId is a Marked Element list. The elements in a Marked Element List may be in any order and not all elements need to be present in every packet. When a producer has a list element that has not changed since the last packet, the producer does not include the element in the current packet's list. Marked Element lists offer flexibility but consume more bytes per element by adding the mimdId.

For example, a producer chooses to use a Marked Element list for a Platform instance's sensor list containing four sensors in its full-list, S_1, S_2, S_3, and S_4. Figure 13 illustrates a Marked Element Sensor List (DLP) with the first element being the AdminPack. The figure includes the AdminPack byte's bit addresses and an indication of the byte's most significant bit (msb) and least significant bit (lsb). The AdminPack's Admin Bit Flags has the List Organization bit set to 0 to indicate the list is a Marked Element list; the next six bits are future values all set to zero. Therefore, the Administration Pack is a one-byte value of 0x00. Following the List Administration Pack, the producer includes the packet-list Sensor elements, which in this example are two of the four sensors: S_1 and S_3. With Marked Element lists each element must include their mimdId values; in this example S_1 (0x04) and S_3 (0x17). This example shows the list elements' do not need to be in the full-list's order, i.e., S_1 is after S_3 in the list.

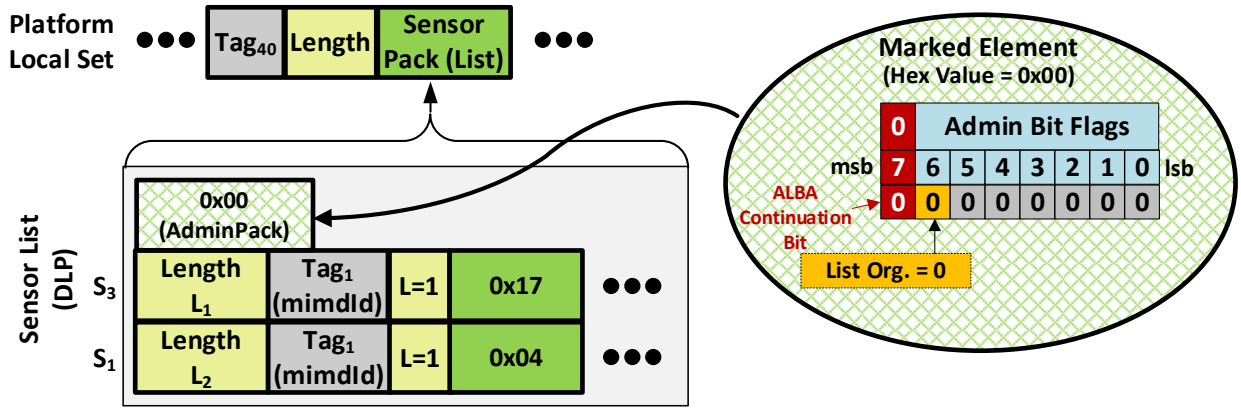


Figure 13: Marked Element List Example

When producers have a Marked Element list element they need to denote as ROC_Unknown, the producer includes the item in the list, with its mimdId value, and sets the ROC Bit Mask's bit for that list position to one. Figure 14 repeats the previous example but sets S_4 to ROC_Unknown. The illustration shows the List Series containing three elements, S_3, S_4, and S_1; S_3 and S_4 provide data as per the previous example, but S_4 only includes its mimdId (0x16). The first byte of the AdminPack's ALBA value includes an ALBA continuation bit (red) set to one, meaning there is another ALBA byte to process, which is the ROC Bit Mask (pink). The second byte is the start of the ROC Bit Mask after the ALBA continuation bit, which is set to zero since this is the last byte of the AdminPack. The ROC Bit Mask has its second bit set to one, indicating the second element of the List Series is to be set to ROC_Unknown by the reader after processing any data for that element; in this example, there is no additional data to process for S_4. The resulting AdminPack value is 0x8020.

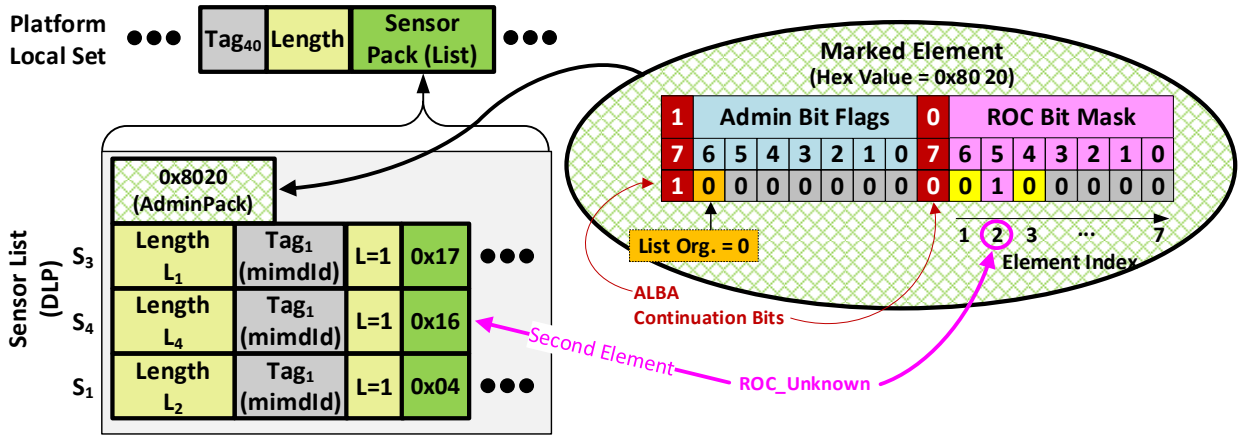


Figure 14: Example of Setting a Marked Element List Element to ROC_Unknown.

With Marked Element lists the packet-list can be in any order, so producers can reorder the elements for ROC_Unknown to be at the beginning of the list to potentially reduce the size of the ROC Bit Mask. For example, if a list has 100 elements and only element 91 is changing to ROC_Unknown, then when the producer is forming the packet-list, the producer places element 91 as the first element in the list; therefore, when the producer constructs the ROC Bit Mask, only one byte is needed.

6.1.2.1.1.2 Fixed Order Lists

ST1901 defines Fixed Order Lists using the fixed-full-list and the fixed-packet-list. Producers write the elements of a Fixed Order List in the same order in every packet, without the need for the mimdId. When a producer has a list element that does not change since the last packet, the producer fills the list elements position with a Zero Length Element (ZLE). A ZLE is an element in the List Series with a length of zero and no data following the length. The ZLE is the method of providing a "filler" (see ST 1901) element for the Fixed Order list.

For example, a producer chooses to use a Fixed Order list for a Platform instance's sensor list containing four sensors in its fixed-full-list, S_1, S_2, S_3, and S_4. The producer is only updating fixed-full-list elements 1 and 3, so elements 2 and 4 are set to ZLE to indicate there is no change to these elements. Figure 15 illustrates this example with the DLP's first element being the AdminPack. The figure includes the AdminPack byte's bit addresses and an indication of the byte's most significant bit (msb) and least significant bit (lsb). The AdminPack's Admin Bit Flags has the List Organization bit (orange) set to 1, indicating a Fixed Order list; the next six bits are future values all set to zero (grey). Therefore, the Administration Pack is a one-byte value of 0x40. The producer sets the List Series first element to S_1's length and data; second element to ZLE, indicating S_2 has no change to its data since the last packet; third element to S_3's length and data; and fourth element to ZLE, indicating S_4 has no data change to its data since the last packet.

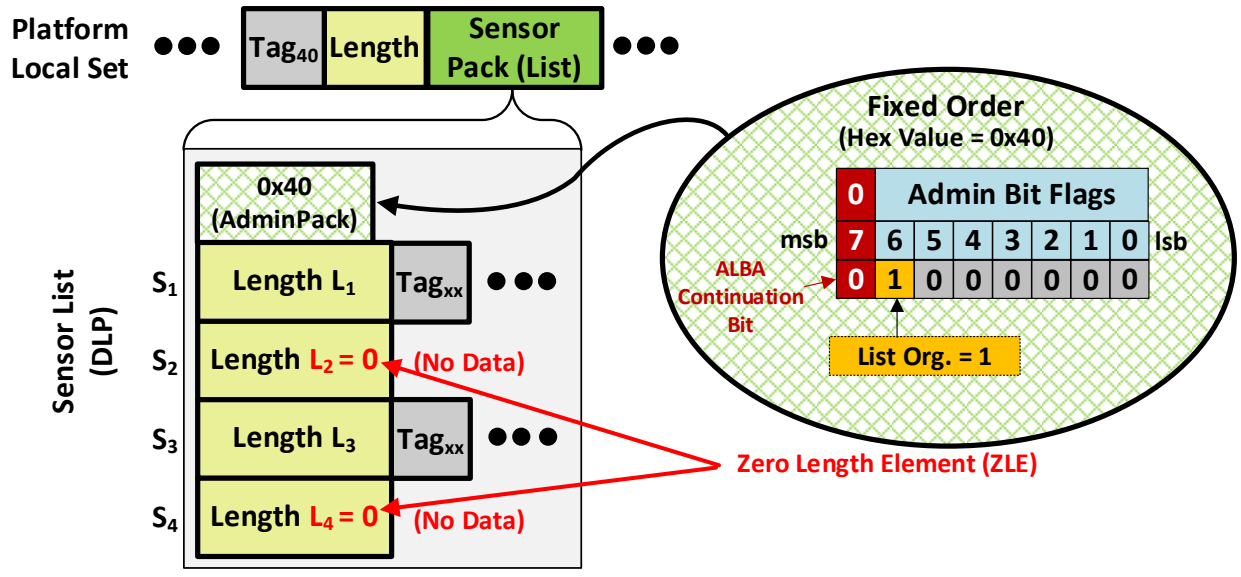


Figure 15: Fixed Order List Example 1

When producers have a Fixed Order list element they need to mark as ROC_Unknown, the producer includes the item in the packet-list (or uses a ZLE) in the correct list position and sets the ROC Bit Mask's bit for that list position to one. Figure 16 repeats the previous example but sets S_3 to ROC_Unknown. The illustration shows the List Series containing four elements, S_1 to S_4; S_1 and S_4 provide data; S_2 and S_3 are ZLE. The first byte of the AdminPack's ALBA value includes a ALBA continuation bit (red) set to one, meaning there is another ALBA byte to process, which is the ROC Bit Mask (pink). The second byte is the start of the ROC Bit Mask

after the ALBA continuation bit, which is set to zero since this is the last byte of the AdminPack. The ROC Bit Mask has its third bit set to one, indicating the third element of the List Series is to be set to ROC_Unknown by the reader after processing any data for that element; in this example, there is no additional data to process for S_3 (i.e., its set to ZLE). The resulting AdminPack value is 0xC010.

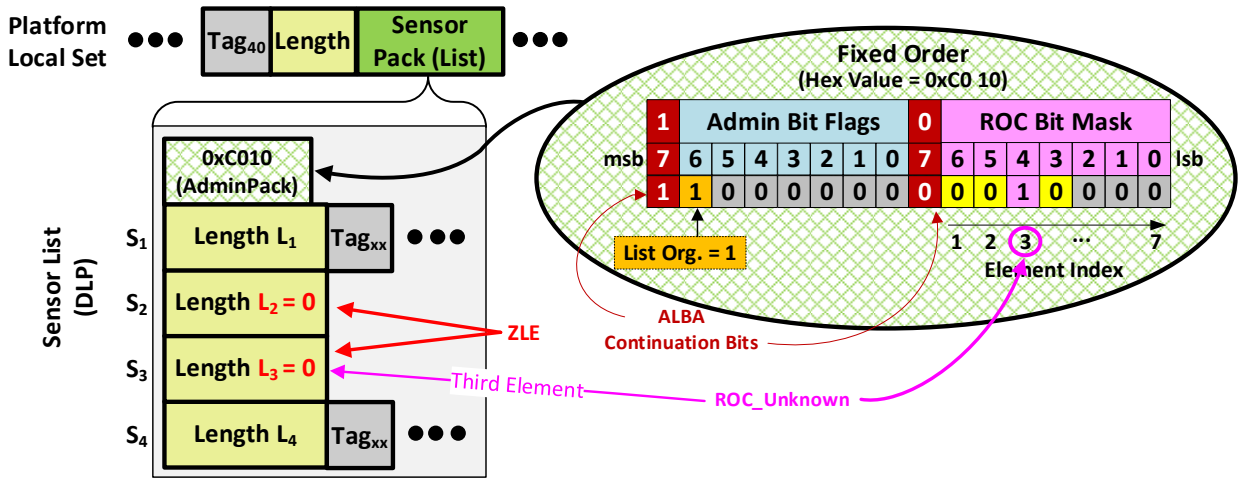


Figure 16: Example of Setting a Fixed Order List Element to ROC_Unknown.

With Fixed Order lists the producer must maintain the ROC_Unknown element's position in the fixed-full-list to retain the original list order when sending subsequent packet-lists for the entire MIMD Stream. Producers "hold" the position in the packet-list using the filler element, i.e., a ZLE.

6.1.3 Inheritance Transmutation

An Inheritance relationship in a class model becomes a single item in the parent class's Local Set; the single item's value is the "value" portion of a Local Set with its constituent items. This is the same method as a Composition relationship (see Section 6.1.2), except an Inheritance relationship does not allow multiplicity and the parent only allows one inherited child per parent instance. A child class does not define its own MIMD Base Attributes; instead, the child's Base Attributes are the same as the parent's Base Attributes. An Inheritance relationship allows a child class to add more specificity to the more generalized parent. A model which includes two or more child classes inheriting from a parent provides an "exclusive or" relationship between the children; either the parent class is using child class 1, or child class 2, etc. but not both. Per requirements in ST1901, once a producer defines an inheritance relationship instance, the relationship does not change for the entire MIMD Stream.

The MIMD model defines inheritance-children as attributes within the parent class. When instantiating an inheritance parent/child relation, the producer treats the pair as one inheritance instance; therefore, if a producer needs to "Zero Length Item" (ZLI) the inheritance pair, the producer uses a ZLI for the parent, i.e., producers do not use ZLI on the child. With multiple parent/child inheritance relationships, the top-most inheritance parent is where the producer uses a ZLI.

Figure 17 illustrates inheritance transmutation.

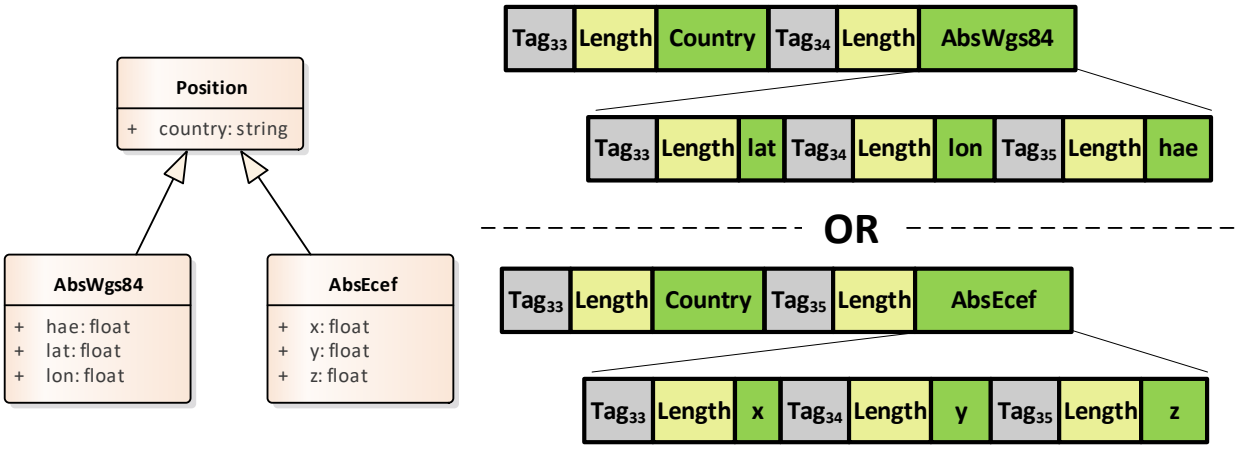


Figure 17: Illustration of Inheritance

The model on the left shows the Position class is the superclass for both child classes AbsWgs84 and AbsEcef. For a given instance of Position producers may include only one child in the Position Local Set. The right side of the figure shows the two possible Local Sets for Position with an inherited child; the top Local Set uses AbsWgs84 (using tag 34) and the bottom Local Set uses AbsEcef (using tag 35). Producers may not create a Position Local Set including both AbsWgs84 and AbsEcef.

6.1.4 Directed Association (Reference) Transmutation

Directed Associations may be singular (REF<ClassName>) or an array (REF<ClassName>[]).

A single Directed Association or Reference ("REF") is a mimdId value with a two-value Tuple Type. A single Directed Association value transmutes using the same instructions as the Tuple Type in Section 6.1.1.1.7 with the first value equal to the serial identifier and the second value equal to the group identifier. If the group identifier is zero it may be removed from the result.

6.1.4.1 Transmuting an Array of Directed Associations

An array of Directed Associations, or references, transmutes to a list of BER-OID values. Directed Associations or References are mimdIds with a two-value Tuple Type. When transmuting an array of Directed Associations, each mimdId transmutation must contain both the serial number and the group identifier, even if the group identifier is the default group value (zero).

A receiver interprets the list as pairs of values, with the first value of each pair being the serial number, and the second value of the pair being the group id.

The transmutation of an array of Directed Associations transmutes each mimdId, using the instructions in Section 6.1.4, but producers do not remove a group identifier equal to zero. The resulting values combine into one list of BER-OID values. The Local Set's item length is the number of bytes for the combined list of BER-OID values

An example transmutation of a Directed Association array with three elements is: the first reference has instance serial number = 100 and group identifier = 200; the second reference has serial number = 10 and group identifier= 0; and the third reference has instance serial number = 400 and group identifier = 160000. This becomes a six element UInt list containing the values

[100, 200, 10, 0, 400, 160000], which producers transmute into the six BER-OID values [0x64, 0x8148, 0x0A, 0x00, 0x8310, 0x89E200], resulting in a single KLV item's value: 0x6481 480A 0083 1089 E200. The KLV item's length for this example is ten bytes.

The decoding process repeatedly decodes individual BER-OID values in pairs from the KLV item's value without exceeding the KLV item's length.

6.2 Class based Transmutation

Future revisions of this standard will support class by class-based transmutation to take advantage of existing MISB Standards. For example, the SDCC class would use MISB ST 1010 to compact the data efficiently. The method of signaling when to use Class based transmutation is under investigation.

6.3 MIMD Packet Instructions

A MIMD Packet contains a KLV key, packet length, and payload. Figure 18 illustrates the three parts of the packet, the KLV key (blue) is the MIMD KLV Defined Length Pack key. The packet length (yellow) is the length of the payload (green). The payload contains error detection values along with the MIMD data.

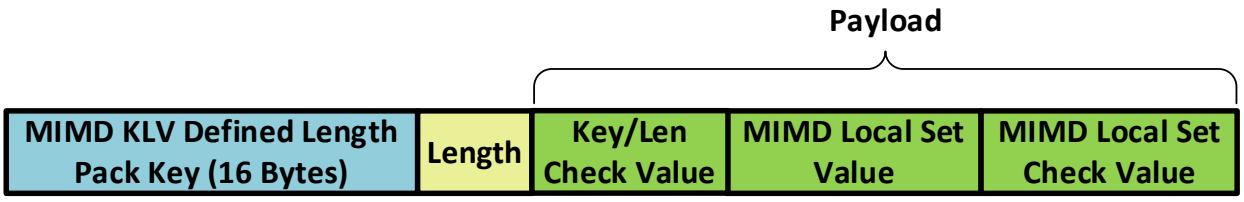


Figure 18: MIMD Packet (DLP)

6.3.1 MIMD KLV Key

The MIMD KLV Defined Length Pack Key is a 16-byte Universal Label identifying the MIMD Packet as a Defined Length Pack (DLP). The MISB registers the MIMD KLV DLP Universal Label in MISB ST 0807 [12] as:

06.0E.2B.34.02.05.01.01.0E.01.05.06.00.00.00.00 (CRC 4670)

The CRC listed with the key above provides a means to validate the key copies correctly (i.e., into documents or into software) – this CRC is **not** a part of the MIMD packet data.

The payload contains a value to validate the key and length portion of the packet, see Section 6.3.4.

6.3.2 Length

The packet length is the ASN.1 BER length encoding of the payload size. The payload contains a value to validate the key and length portion of the packet, see Section 6.3.4.

6.3.3 Payload

The payload is a defined length pack with three elements in the following order: Key/Len Check Value, MIMD Local Set Value, and MIMD Local Set Check Value. The check values (first and last elements) are fixed length. The MIMD Local Set Value is variable length.

1. The Key/Len Check Value contains the result of computing a Cyclic Redundancy Check (CRC) check algorithm on the combined Key and Length bytes. The Key/Len Check Value is a fixed size of one byte. Section 6.3.4 provides more information on computing and using this value.
2. The MIMD Local Set Value is the KLV encoding of the MIMD class instance and all its child instances resulting from applying the MIMD Model-to-KLV Transmutation Instructions to the MIMD class instance. The MIMD Packet only includes the value portion of the MIMD Class transmutation; that is, there is no leading Tag nor Length for the MIMD class itself in the MIMD Local Set Value. When parsing a MIMD packet, receivers compute the length of the MIMD Local Set Value by subtracting three (three bytes for the check values) from the length value.
3. The MIMD Local Set Check Value is the result of computing a CRC on the MIMD Local Set Value. The MIMD Local Set Check Value is a fixed size of two-bytes. Section 6.3.4 provides more information on computing and using this value.

The input into the MIMD check algorithm's is different from other MISB standards (e.g., MISB ST 0601); the MIMD Key/Len Check Value is not a Local Set item inside the data to validate, so the MIMD CRC checks are easier to implement than other MISB standards.

6.3.4 Packet Processing

The MIMD packet provides two different check values to aid in processing and validating the MIMD data: Key/Length Check Value and MIMD Local Set Check Value. Producers use the Key/Length Check Value to validate the combination of the 16-byte key and the packet length. This ensures no corruption of the key occurred during transmission. If there is corruption in the packet length value, the subsequent read operation of the payload data will read the wrong amount of data for the payload. A serious possibility is this subsequent read operation will try to read very large amounts of data, which would affect the system's performance. Figure 19 illustrates the computation of the Key/Length Check Value using the Key and Length of the packet.

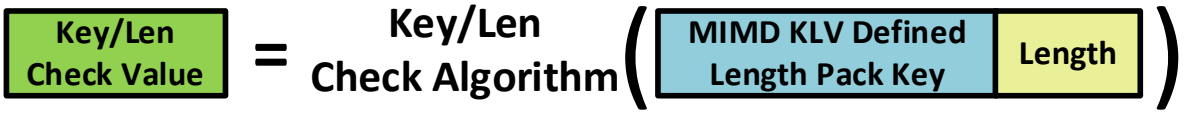


Figure 19: Key/Length Check Value

The Key/Len Check Algorithm is the CRC-8-CCITT which produces the one-byte Key/Len Check Value. This CRC performs well for message bit strings up to 247 bits, which easily covers the 16-byte key and additional length bytes. The Motion Imagery Handbook provides further details of the CRC-8-CCITT algorithm in an appendix.

Producers use the MIMD Local Set Check Value to validate the MIMD Local Set Value.

As shown in Figure 20, the MIMD Local Set Check Value is the result of the MIMD Local Set Check Algorithm on the MIMD data bytes. The MIMD Local Set Check Algorithm is the CRC-16-CCITT, which produces a two-byte MIMD Local Set Check Value. The Motion Imagery Handbook lists the CRC-16-CCITT algorithm in an appendix.



Figure 20: MIMD Local Set Check Value Computation

Requirement(s)	
ST 1902-07	Producers shall compute the CRC-16-CCITT value in accordance with the algorithm in the MISB Motion Imagery Handbook.
ST 1902.2-09	Producers shall compute the CRC-8-CCITT value in accordance with the algorithm in the MISB Motion Imagery Handbook

7 Change Management

As MISB ST 1902 evolves there are two types of possible changes to the MIMD Packet and the MIMD KLV-to-Model Transmutation Instructions: Compatible and Incompatible. Compatible changes are updates to the standard where an existing implementation will continue to function but without necessarily utilizing a newly added capability; these are "backward compatible" changes. An example is adding a new data type to the model which requires an update to the transmutation rules. In this case, MISB ST 1901 requires receivers to ignore unknown [to the receiver] attributes, so the new data type does not affect an existing receiver processing newer metadata.

Incompatible changes are updates to the standard where an existing implementation will not process the data correctly. Examples of incompatible changes include using a different check value algorithm or changing the method of processing arrays. When incompatible changes occur in the standard, the updated standard will define a new MIMD KLV Key for the MIMD Packet.

8 Retired Requirements

The following requirement has been retired from this standard.

Requirement	
ST 1902-02 (Retired)	Where a String attribute includes the MaximumLength trait, an encoded UTF-8 character string shall have less than or equal to the traits MaximumLength number of characters. *(Duplicate of ST1901 requirement)*
ST 1902-03 (Retired)	Integer attributes that define a valid range with their Minimum and Maximum traits shall have encoded values within the range, inclusively. *(Duplicate of ST1901 requirement)*
ST 1902-04 (Retired)	UInt attributes that define a valid range with their Minimum and Maximum traits shall have encoded values within the range, inclusively. *(Duplicate of ST1901 requirement)*
ST 1902-05 (Retired)	Where producers dynamically compute Minimum and Maximum values for an MDAP Array, the computed Minimum and Maximum values shall not exceed the trait's Minimum and Maximum values.
ST 1902-06 (Retired)	Where a class has two or more inheriting child classes, a producer shall only encode one inheriting child class in the parent class Local Set.

Appendix A - Arbitrary Length Bit Array (ALBA)

An Arbitrary Length Bit Array (ALBA) is a MISB data construct for encoding a dynamically sized bit-array. A bit-array is a series of bits, starting with bit zero and ending with bit N. Figure 21 illustrates a bit array with bits b_0 to b_N.

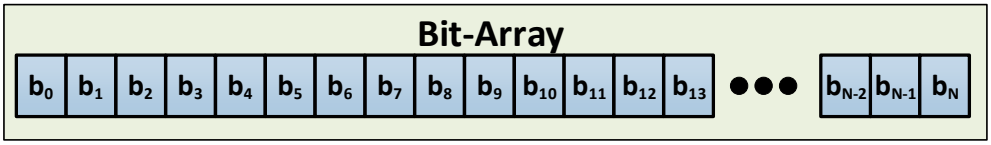


Figure 21: Bit-Array

An Arbitrary Length Bit Array (ALBA) is a series of one or more bytes, where the most significant bit (msb) of each byte signals if more bytes follow in storing the bit-array. The msb of each ALBA byte is the ALBA continuation bit. If the ALBA continuation bit is one, the next byte continues the bit array; if the ALBA continuation bit is zero, the byte is the last byte of the array. Combining the remaining seven bits of each byte creates the original bit-array. ALBA store bit-arrays in 7-bit chunks, starting with chunk zero and ending with chunk M. If N is not a multiple of seven, the producer pads the ALBA bit-array with extra bits (set to zero) after bit N, until the number of bits is a multiple of seven. M derives from N using, $M = \left\lceil \frac{N}{7} \right\rceil$, where the $\lceil\ \rceil$ is the ceiling function. Producers do not specifically transmit the bit-array's full-length value, N+1, i.e., the context where and how a standard uses ALBA provides the method to determine the length or the standard will specify how the producer transmits the length by other means (e.g., another item). The receiver uses the context or known length to determine which bits are part of the bit-array and which bits are padding.

Within the original bit-array, the largest index of the bits set to 1, is the max-index. An ALBA optimization is to only send the bytes up to and including the max-index, i.e., producers do not need to transmit the ALBA bytes after the max-index. When using this optimization, the byte with the max-index will have its continuation bit set to zero.

Because of the possible truncation of trailing bytes, standards and systems should not use ALBA to transmit integer values. For unsigned integers, standards and systems should use BER-OID values.

Figure 22 illustrates a one-byte ALBA example, where the most significant bit (green), or ALBA continuation bit (i.e., bit 7), is set to zero, indicating there are no further bytes needed to represent the bit-array. In this example the resulting bit-array is seven bits, b_0 to b_6.

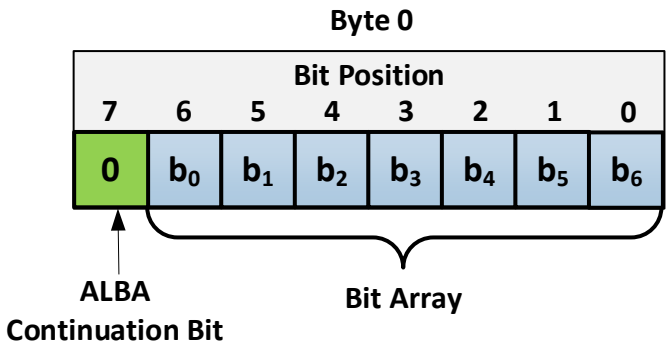


Figure 22: One-byte ALBA Example

With multi-byte ALBA values, each byte has the ALBA continuation bit set to one, except for the final byte, where the ALBA continuation bit is set to zero. Figure 23 shows, when the ALBA continuation bit is one (green), there is at least one more byte of data. An ALBA continuation bit value of zero (red) signals the end of the ALBA bytes. Combining the remaining seven bits of each byte creates the bit-array plus any padding (yellow).

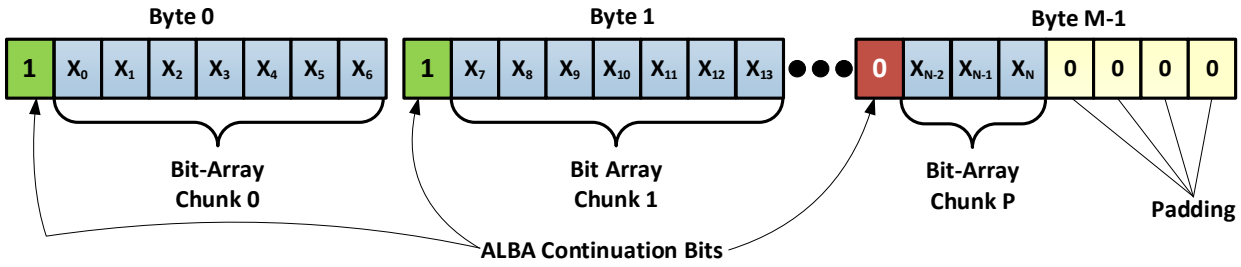


Figure 23: Multi-byte ALBA Example

Implementation Note: BER-OID encoding and ALBA use the same storage technique (i.e., using a continuation bit), however BER-OID assumes the resulting bits form an integer. BER-OID requires the removal of all leading most significant "zero" bytes (i.e., 0x80) from the BER-OID value while ALBA does not have this requirement; ALBA needs the bits in these bytes to maintain a bit position count.